Day 3: Helm & Kubernetes Security (RBAC)

Welcome to Day 3 of the Containers & Kubernetes workshop! Today we explore two critical topics: **Helm** – the package manager for Kubernetes – and **RBAC (Role-Based Access Control)** – the foundation of Kubernetes security. By the end of the day, you will:

- Understand Helm's architecture and benefits.
- Install, upgrade, and rollback applications using Helm charts.
- Create your own Helm chart and customise deployments with values.
- Grasp the security challenges in Kubernetes and how RBAC addresses them.
- Define Roles, ClusterRoles, and bind them to users and ServiceAccounts.
- Apply least-privilege principles to secure your cluster.

Prerequisites

- Docker and Minikube installed and running (from Days 1–2).
- `kubectl` configured to talk to your Minikube cluster.
- Helm installed (see pre-requisites setup). Verify with:

```
helm version
```

- Basic understanding of Kubernetes objects (Pods, Deployments, Services).

09:00 – 10:30 – Conceptual Session: Helm Fundamentals

1.1 What is Helm?

Analogy

Think of Helm as **apt** (for Debian) or **yum** (for Red Hat) but for Kubernetes. It packages multiple Kubernetes YAML files into a single unit called a **chart**. A chart can be easily shared, versioned, and deployed with a single command.

Core Concepts

- **Chart**: A bundle of YAML templates and configuration that defines a Kubernetes application.
- **Release**: A specific instance of a chart deployed to a cluster. You can have multiple releases of the same chart (e.g., `my-app-dev`, `my-app-prod`).
- **Repository**: A collection of charts that can be searched and downloaded.

Why Helm?

- **Reusability**: Package and share your application.
- **Complexity Management**: Templates allow you to parameterise configurations.
- **Easy Updates**: `helm upgrade` handles rolling updates and rollbacks.
- **Dependency Management**: Charts can depend on other charts.

Diagram: Helm Workflow



1.2 Helm Architecture (v3)

Helm v3 is a single binary client; there is no server-side component (Tiller was removed in v3). The client interacts directly with the Kubernetes API server, using your `kubeconfig` for authentication.

Chart Structure

When you create a chart with `helm create`, you get this directory layout:

```
mychart/
├── Chart.yaml          # Metadata: name, version, description
├── values.yaml         # Default configuration values
├── charts/             # Directory for dependent charts (if any)
└── templates/          # Template files that generate Kubernetes manifests
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    └── _helpers.tpl    # Reusable template snippets
```

- **`Chart.yaml`**: Required. Contains chart name, version, API version, etc.
- **`values.yaml`**: Default values that can be overridden during install/upgrade.
- **`templates/`**: Go template files that combine with values to produce YAML.
- **`charts/`**: Stores chart dependencies (when using `helm dependency`).

Template Functions

Helm uses the Go templating engine. Common functions:

- `.Values.<key>` – access values from `values.yaml`.
- `.Release.Name` – the release name.
- `.Chart.Name` – chart name.
- Template directives: `{{ if ... }}`, `{{ range ... }}`, `{{ include }}`, `{{ default }}`.

1.3 Helm Commands

Command	Description
`helm repo add <name> <url>`	Add a chart repository.
`helm repo update`	Update local cache of repository indexes.
`helm search repo <keyword>`	Search for charts in added repositories.
`helm search hub <keyword>`	Search the Artifact Hub (public repository of charts).
`helm install <name> <chart>`	Install a chart, creating a new release.
`helm upgrade <release> <chart>`	Upgrade an existing release to a new chart version.
`helm rollback <release> <revision>`	Roll back to a previous release revision.

Command	Description
`helm history <release>`	List release revisions.
`helm list`	List all releases in the current namespace.
`helm uninstall <release>`	Delete a release (and its associated resources).
`helm create <name>`	Create a new chart skeleton.
`helm package <chart>`	Package a chart into a versioned archive file.
`helm dependency update`	Update dependencies from `Chart.yaml`.

Demo: Install nginx from Bitnami repo

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm install my-nginx bitnami/nginx --set service.type=NodePort
```

This downloads the nginx chart, deploys it, and returns a release named `my-nginx`.

1.4 Creating Custom Charts

Why create your own chart?

- Standardise deployments of your applications.
- Share with your team or the community.
- Manage complex applications with multiple components.

`helm create myapp` generates a template chart with sensible defaults. You then modify the templates and `values.yaml` to match your app.

Values and Overrides

The `values.yaml` file contains default values. When installing, you can override them:

- `--set key=value` – override individual values.
- `-f myvalues.yaml` – override using a custom values file.

Templates

Inside templates, you use expressions like `{{ .Values.replicaCount }}`. During installation, Helm substitutes the actual values and renders the final YAML.

Diagram: Helm Templating

```
values.yaml
+--------------------+
| replicaCount: 3    |
| image: nginx       |
+--------------------+
          |
          v
templates/deployment.yaml
+---------------------------------+
| apiVersion: apps/v1             |
| kind: Deployment                |
| spec:                           |
|   replicas: {{ .Values.replicaCount }} |
|   template:                     |
|     spec:                       |
|       containers:               |
|       - image: {{ .Values.image }} |
+---------------------------------+
          |
          v
Rendered YAML sent to Kubernetes
+---------------------------------+
| apiVersion: apps/v1             |
| kind: Deployment                |
| spec:                           |
|   replicas: 3                   |
|   template: ...                 |
|     spec:                       |
|       containers:               |
|       - image: nginx            |
+---------------------------------+
```

10:45 – 12:30 – Hands-on Lab 1: Helm Basics

Perform each exercise step by step. Keep your terminal open.

Exercise 1.1 – Add Repositories and Search

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx   # search in local repos
helm search hub nginx    # search Artifact Hub (public)
```

Exercise 1.2 – Install a Chart

```
helm install my-nginx bitnami/nginx --set service.type=NodePort
```

Check release status:

```
helm list
kubectl get all -l app.kubernetes.io/instance=my-nginx   # depends on chart labels; alternatively just get all
```

Access via Minikube:

```
minikube service my-nginx   # opens browser
```

Exercise 1.3 – Upgrade and Rollback

Upgrade to increase replicas:

```
helm upgrade my-nginx bitnami/nginx --set replicaCount=3
```

Check:

```
kubectl get pods
```

View history:

```
helm history my-nginx
```

Rollback to revision 1:

```
helm rollback my-nginx 1
```

Verify that replicas returned to original (likely 1).

Exercise 1.4 – Create Your First Chart

```
helm create myapp
cd myapp
tree .   # explore structure
```

Examine `Chart.yaml`, `values.yaml`, and the templates.

Install the chart (with default values):

```
helm install test-app .
```

List resources:

```
kubectl get all -l app.kubernetes.io/instance=test-app
```

Exercise 1.5 – Customize Values

Edit `values.yaml` (or create a separate `myvalues.yaml`). For example, change:

```
replicaCount: 2
image:
  repository: nginx
  tag: latest
service:
  type: NodePort
  port: 80
```

Upgrade the release:

```
helm upgrade test-app . -f values.yaml   # or use --set
```

Check that replicaCount changed.

Exercise 1.6 – Uninstall

```
helm uninstall my-nginx test-app
```

Confirm with `helm list`.

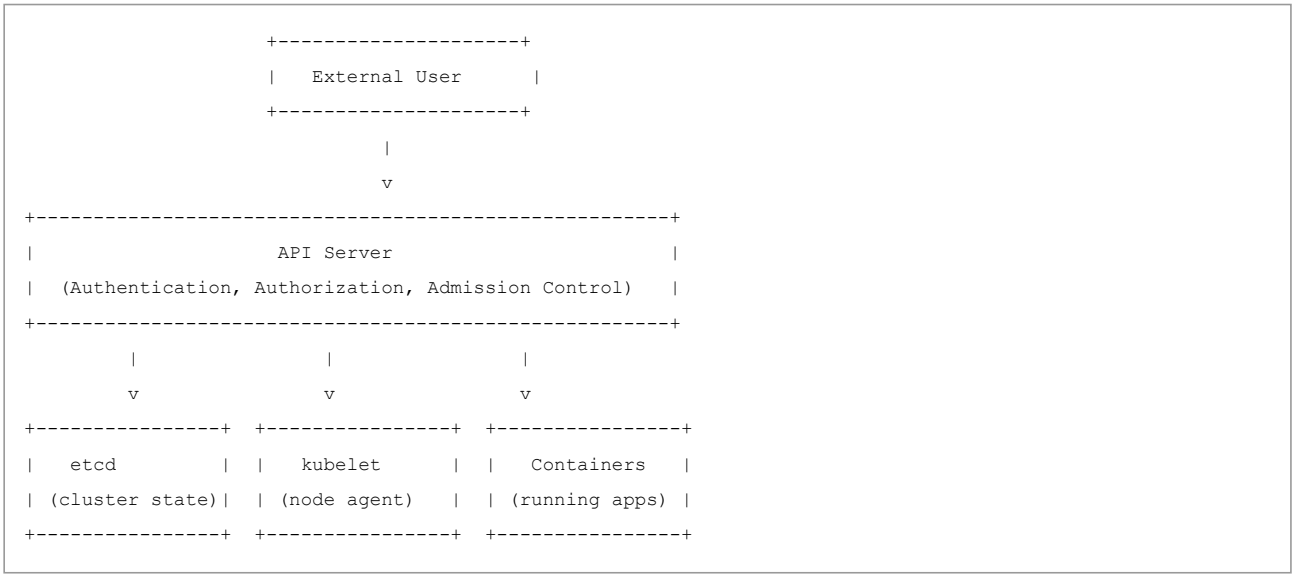
13:30 – 15:00 – Conceptual Session: Kubernetes Security & RBAC

2.1 Kubernetes Security Challenges

Kubernetes clusters have a large attack surface. Common real-world incidents include:

- **Misconfigured RBAC**: Giving too many permissions (e.g., Tesla cryptojacking incident where an exposed dashboard allowed attackers to deploy miners).
- **Exposed API server**: Allowing unauthenticated access to the API server.
- **Vulnerable container images**: Running images with known vulnerabilities.
- **Compromised etcd**: If etcd is not secured, the entire cluster state is exposed.

Attack Surface Diagram



Each component must be secured.

2.2 Authentication & Authorization

Authentication – verifying who you are. Kubernetes supports:

- **Client certificates** (X.509)
- **Bearer tokens** (static tokens, service account tokens)
- **OpenID Connect (OIDC)** – integration with identity providers
- **Webhook token authentication** – external service validates tokens

Authorization – determining what you can do. The primary mode is **RBAC** (Role-Based Access Control). ABAC (Attribute-Based Access Control) is legacy and complex.

RBAC Flow



2.3 RBAC Objects

RBAC has four key objects:

Kind	Scope	Description
Role	Namespace	Defines a set of permissions (verbs on resources) within a single namespace.
ClusterRole	Cluster-wide	Defines permissions that can be used across all namespaces or on cluster-level resources (e.g., nodes).
RoleBinding	Namespace	Grants a Role to a user, group, or ServiceAccount within a specific namespace.
ClusterRoleBinding	Cluster-wide	Grants a ClusterRole to a user, group, or ServiceAccount across the entire cluster.

Example Role YAML:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]   # core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Example RoleBinding YAML:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Common Verbs: `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`.

ClusterRole can also grant access to cluster-scoped resources like `nodes` or non-resource URLs like `/healthz`.

2.4 Service Accounts

A **ServiceAccount** provides an identity for processes running in a Pod. By default, every namespace has a `default` ServiceAccount, but it's limited.

- When a Pod is created, you can specify a `serviceAccountName`. If not, the `default` is used.
- The ServiceAccount's token is mounted into the Pod at `/var/run/secrets/kubernetes.io/serviceaccount/token`, allowing the Pod to authenticate to the API server.

Custom ServiceAccount – useful for granting specific permissions to an application (e.g., a CI/CD pod that needs to create deployments).

15:15 – 16:45 – Hands-on Lab 2: RBAC Implementation

Perform these exercises to understand RBAC in action.

Exercise 2.1 – Check Current Permissions

First, see what you (the current user) can do:

```
kubectl auth can-i create deployments
kubectl auth can-i delete pods
kubectl auth can-i list pods --as system:serviceaccount:default:default   # check default SA permissions
```

Exercise 2.2 – Create a Role

Create `pod-reader-role.yaml`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Apply:

```
kubectl apply -f pod-reader-role.yaml
```

Exercise 2.3 – Create RoleBinding

Create `pod-reader-binding.yaml`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```
kubectl apply -f pod-reader-binding.yaml
```

Exercise 2.4 – Test with kubectl as User

Now test what user `alice` can do. The `--as` flag impersonates a user (requires your own user to have impersonation privileges, but in a real cluster you'd need proper auth). For learning, we can use `--as` to simulate:

```
kubectl get pods --as alice          # should work
kubectl delete pod any-pod --as alice # should fail (verb not allowed)
```

Exercise 2.5 – Create ClusterRole

Create `secret-reader-clusterrole.yaml`:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: secret-reader
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "list"]
```

Apply:

```
kubectl apply -f secret-reader-clusterrole.yaml
```

Now create a namespace `dev` and bind the ClusterRole to user `bob` in that namespace:

```
kubectl create namespace dev
kubectl create rolebinding bob-secret-reader --clusterrole=secret-reader --user=bob -n dev
```

Test:

```
kubectl get secrets -n dev --as bob     # should work
kubectl get secrets --as bob             # fails because binding is only in dev namespace
```

Exercise 2.6 – Service Account with RBAC

Create a ServiceAccount:

```
kubectl create sa my-sa
```

Bind the `pod-reader` Role to this ServiceAccount:

```
kubectl create rolebinding my-sa-pod-reader --role=pod-reader --serviceaccount=default:my-sa -n default
```

Now create a Pod that uses this ServiceAccount:

```
kubectl run test-pod --image=nginx --serviceaccount=my-sa
```

Wait for the pod to run. Then exec into it and try to access the Kubernetes API:

```
kubectl exec -it test-pod -- /bin/bash
# Inside container:
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
APISERVER=https://kubernetes.default.svc
curl -H "Authorization: Bearer $TOKEN" $APISERVER/api/v1/namespaces/default/pods --insecure
```

Because the ServiceAccount has the `pod-reader` Role, it should list pods successfully. Try to delete a pod with `curl` – it should fail.

Exit the container.

Exercise 2.7 – Cleanup

```
kubectl delete role pod-reader
kubectl delete rolebinding read-pods
kubectl delete clusterrole secret-reader
kubectl delete rolebinding bob-secret-reader -n dev
kubectl delete namespace dev
kubectl delete sa my-sa
kubectl delete pod test-pod
```

Daily Recap & Troubleshooting

Key Takeaways

- **Helm** simplifies Kubernetes deployments through charts, releases, and repositories.
- Charts are structured with `Chart.yaml`, `values.yaml`, and `templates/`.
- Common Helm commands: `install`, `upgrade`, `rollback`, `uninstall`.
- **RBAC** is the primary authorization mechanism in Kubernetes.
- **Role** and **ClusterRole** define permissions; **RoleBinding** and **ClusterRoleBinding** grant them to users, groups, or ServiceAccounts.
- **ServiceAccounts** give Pods an identity for API access.
- Always follow the principle of **least privilege** when assigning RBAC rules.

Common Issues & Solutions

Problem	Likely Cause	Solution
`helm install` fails with "no repository"	Repository not added or not updated.	Run `helm repo add` and `helm repo update`.
`helm upgrade` doesn't change anything	No change in values or chart.	Modify values or chart version; use `--recreate-pods` if needed.
`kubectl auth can-i` returns "no" unexpectedly	RBAC rules not correctly applied.	Check Role/RoleBinding with `kubectl describe`.
Impersonation (`--as`) fails	Your user lacks impersonation privileges.	In a real cluster, you need permissions; for learning, it often works if you're cluster admin.
Pod cannot access API	ServiceAccount token not mounted or permissions insufficient.	Check ServiceAccount, rolebinding, and token path.

Useful Commands Recap (RBAC)

- `kubectl create role <name> --verb=<verbs> --resource=<resources>`
- `kubectl create rolebinding <name> --role=<role> --user=<user>`
- `kubectl create clusterrole <name> --verb=<verbs> --resource=<resources>`
- `kubectl create clusterrolebinding <name> --clusterrole=<role> --user=<user>`
- `kubectl get roles,rolebindings -n <namespace>`

- `kubectl describe role <name> -n <namespace>`
- `kubectl auth can-i <verb> <resource> --as <user>`

Connection to Previous Modules

- **Scripting (Days 36–40)**: Helm commands and kubectl can be scripted for CI/CD.
- **Infrastructure (Days 45–47)**: RBAC is analogous to IAM policies in cloud (AWS IAM, Azure RBAC).
- **Security (Days 49–50)**: IAM concepts (users, groups, policies) directly translate to Kubernetes RBAC (users, groups, roles/bindings).
- **DevSecOps (Days 26–30)**: Security scanning of images and CI/CD pipelines now integrate with Helm and Kubernetes.

Next Steps

Tomorrow (Day 4) we will cover **Monitoring, Logging & Autoscaling**. You'll install Prometheus and Grafana, set up alerts, implement Horizontal Pod Autoscaling (HPA), and apply production best practices. Keep your cluster ready!

Congratulations on completing Day 3! You now have powerful skills to package applications with Helm and secure your cluster with RBAC.